Census Income Classification
&
Customer Segmentation

Project Report

Prepared by: Avleen Mehal

Date: April 2026

Dataset: US Census Bureau - 199,523 records, 1994–95

1. Problem Understanding

The goal is to build a model that identifies individuals likely to earn more than $50,000 per year, so a retail client can target their premium marketing campaigns more effectively. There are two parts:

- A classification model to score each individual, and
- A segmentation model to understand who these people are, so messaging can be tailored.

The client has a fixed campaign budget. A good model does not get the highest accuracy; instead, it helps to spend that budget on the right people.

2. Data Exploration

First look

The dataset has 199,523 rows and 42 columns which is a mix of:

- Numerical features (age, wages, capital gains, weeks worked)
- Categorical features (education, occupation, marital status, country of birth).

The label column had a trailing period that needed to be stripped before anything else.

187,141 negative cases (≤$50k) vs 12,382 positive cases (>$50k).

That's a 15:1 ratio, a 6.2% positive rate. This imbalance ruled out accuracy as a meaningful metric, since a model that predicts "≤$50k" for everyone achieves 93.8% accuracy yet is completely useless.

We will use PR-AUC as the primary metric. Will also tune the classification threshold rather than using the default 0.5.

Missing values

The census encodes missing values as **'?'** rather than leaving them blank.

- Eight columns had them. Four migration columns - each had exactly 99,696 missing rows, which is 49.97% of the dataset. The reason for this identical count across all four could be structural, that is, one census year simply didn't collect migration data.

  On checking their association with the target (Cramer's V between 0.029 and 0.039), the decision was to drop all four columns. High missingness plus a near-zero Cramer's V signal justifies dropping these columns.
- The remaining four missing columns (country of birth for father/mother/self, state of previous residence) were missing at low rates (0.35%–3.4%) with no structural pattern.

  I imputed them as "Unknown" rather than dropping.

What the features tell us

I computed **point-biserial correlation** for every numerical feature and **Cramer's V** for every categorical feature against the target.

Numerical features:

- *weeks worked in year* (r=+0.26) was the clearest separator.
    - The >$50k group concentrates at 52 weeks (full-year employment) while ≤$50k spreads broadly, including many zeros
- *capital gains* (r=+0.24) and *dividends from stocks* (r=+0.18) are 96% and 89% zeros, respectively.
- *wage per hour* (r=+0.024) was surprisingly weak. Non-zero rows actually had a lower positive rate than the population average. Salaried workers dominate the >$50k class and show up as wage=0 in the data
- *year* had only two values (94/95) with nearly identical positive rates, hence dropped.

Categorical features:

- *detailed occupation recode* (V=0.44) and *education* (V=0.39) were the strongest signals by a wide margin
- *Marital status* showed a striking gap: married-civilian-spouse-present had an 11.4% positive rate vs 1.3% for never married - nearly a 9x difference
- *sex* showed a 4x gap (male 10.2%, female 2.6%), which reflects the 1994–95 labor market structure

3. Preprocessing Decisions

What I dropped and why

Column	Reason
4 migration columns	50% missing, Cramer V < 0.04 - structural gap, not recoverable
weight	Census sampling weight - a property of the survey design, not the individual
year	Two values only, r=+0.015, no usable signal

Feature engineering

The financial columns, including *dividends, capital gains/losses,* being 89–98% zeros, were the key insight from EDA.

Applying scaling directly to a column that's 96% zero produces a feature that carries almost no information for most rows.

I created five binary flags:

Flag	Correlation with target
has_dividends	r = +0.270
is_fulltime_year (weeks == 52)	r = +0.263
has_capital_gains	r = +0.215
has_capital_losses	r = +0.139
has_wage	r = -0.013

These ended up stronger than the raw columns and were among the top features in the final model.

Encoding and scaling

- log1p transform on the four skewed financial columns (skewness 9–28) before scaling - raw values would distort linear models
- RobustScaler (median/IQR) for all numerical features - robust to the outliers present in investment income columns
- OrdinalEncoder for all categorical features rather than one-hot encoding. Columns like country of birth (43 values), state (51 values), and household status (38 values) would have produced hundreds of sparse dummy columns. Tree models handle integer-encoded categories natively

Data split

Stratified 60/20/20 split - train/validation/test.

Stratification preserves the 6.2% positive rate in all three sets. The test set was locked and untouched until the very end. All model selection and threshold tuning happened on the validation set only.

4. Model Selection

I trained four models as a progression from a baseline model to a best-in-class ensemble:

Model	Role
Logistic Regression	Interpretable baseline - coefficients are readable by non-technical stakeholders
Random Forest	Robust ensemble baseline - less sensitive to hyperparameters
XGBoost	Gradient boosting

LightGBM	Primary candidate - fastest on large tabular datasets with many categorical columns

Why not SVM: $O(n^2–n^3)$ training complexity on 120k rows would take hours. Its imbalance handling at 15:1 is also weaker than dedicated loss-weighting in boosting models.

Imbalance handling: I used class_weight='balanced' and scale_pos_weight (= 15.11, the ratio of negatives to positives) rather than SMOTE. Class weighting adjusts the loss function directly - it achieves the same mathematical correction as oversampling without adding synthetic rows to the training set.

How each model trains

Logistic Regression minimises log-loss with gradient descent (lbfgs solver, L2 regularization, C=0.1). With class_weight='balanced', errors on the minority class are penalized more heavily. Converged after ~150 iterations.

Random Forest grows 300 independent trees in parallel. Each tree sees a bootstrap sample of rows and a random feature subset at each split. No loss function is minimized across trees - each split just maximizes Gini purity. Fully parallelizable.

XGBoost builds trees sequentially. Each new tree fits the residuals of the current ensemble - it learns where previous trees were wrong. Uses log-loss with L1/L2 regularization on tree weights. I added early stopping on the validation set (50 rounds of patience).

LightGBM is also sequential boosting but uses leaf-wise growth instead of level-wise - it always splits the leaf with the highest loss reduction first. GOSS (Gradient-based One-Side Sampling) focuses each round on the rows with the largest current errors.

5. Evaluation

Why PR-AUC, not accuracy or ROC-AUC

Accuracy is useless here - the trivial model scores 93.8%. ROC-AUC is misleading: because the false positive rate has 187k negatives in the denominator, even thousands of false positives barely move the curve.

PR-AUC only looks at the positive class. Its random baseline equals the positive rate (0.062), so every point above that is real, measurable lift. It's the right metric for this problem.

Threshold tuning

At a 6.2% base rate, the default 0.5 threshold is poorly calibrated. I tuned the threshold on the validation PR curve to maximize F1.

All four models ended up with optimal thresholds between 0.75 and 0.86. The gain from tuning vs using 0.5 was +0.12 to +0.14 F1 points across all models.

Validation results

Model	Threshold	PR-AUC	ROC-AUC	F1	F1@0.5
LightGBM	0.857	0.6819	0.9516	0.6168	0.4982
XGBoost	0.847	0.6789	0.9529	0.6204	0.4813
Random Forest	0.747	0.6027	0.9435	0.5662	0.4593
Logistic Regression	0.825	0.5456	0.9279	0.5293	0.3899

- LightGBM and XGBoost are essentially tied on PR-AUC (gap of 0.003).
- I selected LightGBM - the gap is negligible and LightGBM trains faster, which matters for iteration.
- Notice Logistic Regression's ROC-AUC of 0.928 looks strong but its PR-AUC of 0.546 reveals it struggles to build a precise high-income segment. That discrepancy is exactly why ROC-AUC alone is misleading here.

Hyperparameter tuning

After identifying LightGBM as the best model, I ran a lightweight **RandomizedSearchCV** to validate the Round 1 parameter choices - 25 combinations, 3-fold stratified CV, same PR-AUC scoring metric. The search space was centred around the Round 1 values.

The best combination improved val PR-AUC only by +0.0006 - well below the 0.003 threshold I set for meaningful improvement.

The Round 1 defaults were validated: the initial choices of learning_rate=0.05, num_leaves=63, max_depth=7 were already close to optimal for this dataset.

Final test set evaluation

Run once, at the end, with the threshold fixed at 0.857 from the validation step.

Metric	Validation	Test
PR-AUC	0.6819	0.6903
Precision	0.640	0.652
Recall	0.600	0.605
F1	0.6168	0.6279

Test scores are marginally better than validation across every metric - **no sign of overfitting.** The model flagged 2,299 people on a 39,905-person test set; 65.2% of those flagged actually earn >$50k. That's a 10.5x lift over random targeting (base rate 6.2%).

6. Customer Segmentation

The classification model answers is this person likely to earn >$50k? It doesn't say anything about who they are or how to reach them. A probability score is useful for ranking, but it treats all high earners as identical. A 45-year-old married professional, and a 28-year-old part-time worker might both score similarly but respond to entirely different messages. Segmentation adds that dimension.

Approach

1. I filtered to adults (age >= 16) before clustering. Children make up 25% of the dataset but have a 0% income rate and "Not in universe" for all employment features - including them would just produce one trivial age-based cluster and four useful ones.
2. With 40 features, K-Means distance calculations become unreliable in high dimensions. I ran PCA first and retained 7 components covering 90.7% of variance, then clustered on that reduced representation.
3. Silhouette score peaked at k=7 (0.437). I **overrode** this to k=5 on business grounds - the silhouette gap between k=7 and k=5 is only 0.026, and a retail team can realistically act on 5 distinct personas. Seven produces overlapping profiles that are hard to name and tend to get ignored in practice.

A note on the *'Weight'* column:

The dataset includes a census sampling weight. Each record represents a different number of people in the actual US population due to a stratified survey design. For classification, I dropped it, because the weight describes the survey design, not the individual.

For segmentation, the situation is different. The weight doesn't affects how representative the reported cluster sizes and income rates are of the real population.

I checked whether ignoring weights materially distorted the results. The mean weight for the ≤$50k class (1,737) and >$50k class (1,796) are nearly identical, which means income isn't systematically over- or under-sampled.

The weighted income rates shift by at most 0.6 percentage points per cluster, and the ranking stays exactly the same.

What weighting does add is population scale: the sample counts in the table below are survey records, but summing weights gives an estimate of the actual US population each cluster represents.

The five segments

Cluster	Persona	Size	Est. Population	Income Rate	Priority
1	Established Breadwinners	69,070	~122M	12.6%	High

0	Working Homemakers	41,767	~74M	5.5%	Medium
4	Early Career & Students	18,382	~36M	0.7%	Low (nurture)
2	Part-Time Workforce	14,627	~23M	6.8%	Medium-Low
3	Mid-Career Independents	5,329	~8M	6.0%	Low-Medium

Cluster 1 - Established Breadwinners is the primary target. Median age 45, married men, working all 52 weeks, 8.6% have capital gains. Income rate 12.6% - 50% above the population average. The LightGBM classifier flags 12.8% of this group, highest of all segments. This cluster represents an estimated 122 million people in the US adult population.

Cluster 0 - Working Homemakers is the second priority. Married women in their early 40s, near full-time employment (median 49 weeks), service/administrative roles. Income rate below average at 5.5%, but the classifier flags 4.4% - that's the actionable subset.

Cluster 4 - Early Career & Students has a 0.7% income rate. Not a current premium target, but worth brand-building for future value.

Combining classification and segmentation

The two models are most powerful together. Segmentation tells you which type of person someone is; the classifier tells you whether that specific person is likely to earn >$50k.

- Filter to Cluster 1 first - highest concentration of high-earners
- Within each segment, sort by classifier probability score and target from the top down
- Adjust the classifier threshold to match the campaign budget - raise it for a tighter, higher-precision list, lower it for broader reach

The threshold sensitivity table shows exactly how this trade-off plays out:

Threshold	Precision	Recall	People Flagged
0.70	55%	68%	~3,100
0.857 (tuned)	65%	61%	~2,300
0.95	78%	36%	~1,100

The model doesn't need to be retrained to adjust this - only the threshold changes.

7. Business Recommendations

On precision vs recall: For a retail campaign with a fixed budget per contact, precision is the priority. A false positive is money spent on the wrong person. The tuned threshold of 0.857 sits on the precision-heavy side of the PR curve deliberately. If the client is running a low-cost digital campaign where the marginal cost per impression is near zero, lower the threshold and accept lower precision in exchange for broader reach.

On segmentation: Lead with *Cluster 1 (Established Breadwinners)* for premium offers. Use *Cluster 0 (Working Homemakers)* with value-oriented messaging and let the classifier identify the worthwhile subset. Don't ignore *Cluster 4* - they're not worth targeting now but represent the next generation of high-earners. For campaign sizing decisions, use the population-weighted estimates rather than raw record counts; the sample sizes reflect the survey design, not actual market size.

On the model: Use the LightGBM probability score, not the binary label. Ranking all prospects by score lets the client cut the list wherever the budget runs out rather than being locked to a fixed threshold.

On the data: This is 1994–95 census data. The model captures the income signal in that population well, but income thresholds, occupation structures, and demographic compositions have shifted. The approach is sound; the specific model should be retrained on current data before production use.

Open questions for the client:

- What is the cost of a false positive (targeting a ≤$50k person) vs a false negative (missing a >$50k person)? This determines the optimal operating threshold
- Should children (age < 16) be scored at all, or filtered out upstream?

8. References

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16.
- Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.
- Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. JMLR, 13, 281–305.
- Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. ICML '06.
- Scikit-learn documentation - ColumnTransformer, OrdinalEncoder, RobustScaler, RandomizedSearchCV
- US Census Bureau (1996). Current Population Survey, 1994–95. - original dataset source